Volatile Cyber Defense (VCD)

A New Security Paradigm: Self-Destruction, Forensic Preservation, and Autonomous Recovery Triggered by Attack Detection

Author: Takeru Yamashita

Keywords: Volatile Cyber Defense, Intentional Crash, Forensics-First Termination, Self-Destructing System, Intrusion Response, Elixir/OTP, Kubernetes, Defense in Depth

Abstract

Modern security design assumes that preventing attacks is the primary goal, delegating the handling of post-intrusion contaminated states to after-the-fact forensic investigation. However, when an attacker breaches the first line of defense, the contaminated execution state persists in the system, providing a foothold for lateral movement, long-term persistence, and evidence destruction.

We propose Volatile Cyber Defense (VCD) as a new defense paradigm to address this problem. VCD is based on the design principle: "when an attack is detected, the process immediately preserves evidence and self-destructs, then recovers in a clean state." Unlike existing approaches such as Moving Target Defense, Phoenix Servers, and Intrusion Tolerance, VCD is unique in simultaneously adopting detection-triggered immediate self-destruction, forensics-first termination, and autonomous recovery as its core design principles — a combination with no precedent in prior research.

This paper demonstrates the technical feasibility of VCD through a multi-layer volatile implementation using Elixir/OTP and Kubernetes at both the process and container levels.

1. Introduction

Modern cybersecurity, as exemplified by Defense in Depth and Zero Trust, is fundamentally oriented toward detecting and blocking attacks while maintaining system continuity. However, these designs fail to adequately address what happens after an attacker has breached the first line of defense.

When an attacker successfully intrudes, the contaminated execution state persists in the system's memory, sessions, and variables. The attacker exploits this state as a foothold for lateral movement, long-term persistence, and evidence destruction. Traditional forensic methods investigate and reconstruct such events after the fact, and have no means to prevent the spread of contamination during an active intrusion.

This research defines this problem as the "contaminated state persistence problem" and seeks a solution at the design level. We propose a new defense paradigm — Volatile Cyber Defense (VCD) — in which a component that detects an attack immediately preserves evidence, self-destructs, and autonomously recovers in a clean state.

VCD draws inspiration from Erlang/OTP's "Let it crash" philosophy and Kubernetes' self-healing mechanisms, applying the concept of "designing for failure" to the security domain. Self-destruction is not a system failure but a designed defensive action, structurally eliminating the attacker's opportunity to exploit the contaminated state.

The advent of the cloud-native era has established the infrastructure needed to realize VCD. Container orchestration technologies such as Kubernetes have automated the disposal and regeneration of processes and containers at the infrastructure level. While "designing for failure" was once costly and risky in the physical server era, the destruction and restart of lightweight containers has become a routine part of modern operations. VCD gives this infrastructure a new security purpose.

The contributions of this paper are as follows:

- We formalize the contaminated state persistence problem and systematize it as Volatile Cyber Defense (VCD)
- We clarify VCD's uniqueness through comparison with MTD, Phoenix Servers, and Intrusion Tolerance
- We demonstrate the technical feasibility of VCD through a multi-layer volatile implementation using Elixir/OTP and Kubernetes

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 details the VCD paradigm and architecture. Section 4 presents the implementation. Section 5 provides discussion. Section 6 concludes.

2. Related Work

2.1 Moving Target Defense (MTD)

Moving Target Defense (MTD) is a security strategy that continuously changes system configurations, network topologies, and IP addresses to dynamically alter the attack surface, making attacker reconnaissance and intrusion more difficult [1]. MTD's primary goal is to prevent attackers from locating their target, making it a proactive approach.

VCD shares with MTD the principle of not assuming static state maintenance, but whereas MTD aims at prevention before an attack, VCD adopts immediate response after attack detection as its core design principle. MTD also does not aim to preserve forensic evidence. The two are not in opposition; they can be combined as a layered defense, with MTD making intrusion difficult and VCD functioning when MTD is breached.

2.2 Phoenix Servers

Phoenix Servers is an infrastructure design pattern that eliminates configuration drift by periodically destroying and rebuilding servers, maintaining a known clean state (Fowler, 2012) [2]. It is a foundational concept of Immutable Infrastructure and aligns well with Kubernetes' container design philosophy.

VCD and Phoenix Servers share the idea of "disposing and returning to a clean state," but differ fundamentally in that Phoenix Servers performs schedule-based regeneration for release and operational management purposes, while VCD adopts detection-triggered immediate self-destruction and forensic evidence preservation as its design principles.

2.3 Intrusion Tolerance

Intrusion Tolerance abandons the goal of preventing all intrusions, instead designing mechanisms to prevent intrusions from causing system security failures [3]. It is based on redundancy, diversity, and Byzantine Fault Tolerance, targeting continued operation under intrusion.

VCD and Intrusion Tolerance both differ from traditional defense models in assuming intrusion will occur. However, whereas Intrusion Tolerance targets continued operation under intrusion, VCD targets intentional cessation upon detection and clean regeneration. Their goals of continuation versus elimination make them conceptually opposite.

2.4 Digital Forensics and Incident Response (DFIR)

DFIR is the practice of collecting and analyzing digital evidence after an incident to uncover the full scope of an attack [4]. Recently, the direction of Proactive Forensics, which emphasizes preparation before incidents occur, has also been proposed.

VCD shares DFIR's goal of evidence preservation, but differs in that VCD has the system autonomously transmit evidence before self-destruction, rather than relying on post-incident collection. VCD complements DFIR by preserving evidence before attackers have an opportunity to destroy it.

2.5 VCD's Uniqueness

From the comparisons above, VCD's uniqueness lies in the combination of three principles:

- Detection-triggered immediate self-destruction (Reactive Intentional Crash)
- Forensics-first transmission before self-destruction (Forensics-First Termination)
- Autonomous recovery and return to a clean state

No prior research combines all three of these as simultaneous design principles. VCD is not in opposition to existing concepts; it is positioned as a paradigm that achieves multi-layered defense in combination with them.

3. Volatile Cyber Defense (VCD)

3.1 Definition of Volatility

In VCD, "volatility" refers to the property by which a process or container that detects an attack or anomaly autonomously erases its execution state and restarts. The term derives from the metaphor of chemical volatility (evaporating without leaving traces), analogously describing how contaminated execution state "evaporates."

While traditional security design assumes "defending and continuing," VCD places "intentionally dying upon detection and cleanly reviving" at its core. This idea transfers Erlang/OTP's "Let it crash" philosophy — treating failures as normal states rather than exceptions and delegating recovery responsibility to higher-level mechanisms — to the security context [5].

Volatility is a designed defense property, not a system vulnerability. Even if an attacker attempts lateral movement or long-term persistence by exploiting contaminated state, the state evaporates, structurally eliminating their opportunity.

3.2 Intentional Crash

In VCD, self-destruction is a designed defensive action, not a failure. A component that detects an attack autonomously terminates, achieving the following:

- No memory, sessions, or variables contaminated by the attacker are inherited
- Lateral movement of contaminated state is structurally blocked
- Long-term persistence after session hijacking is prevented

In implementing self-destruction, minimizing the critical path is an important design principle. To "self-destruct before the attacker," the latency from detection to completed self-destruction must be minimized. VCD limits the processing immediately before self-destruction to only a local file write, delegating heavy processing such as data transfer, persistence, and external coordination to a higher layer or separate component. This keeps the self-destruction critical path to two steps:

```
Anomaly detection → File write → exit
```

The delegated component detects file changes and asynchronously takes over subsequent processing. Since subsequent processing can complete evidence transmission even while the self-destructed component has terminated, both self-destruction speed and evidence preservation reliability are achieved.

3.3 Forensics-First Termination

VCD's self-destructing component must transmit forensic information externally before terminating. This design principle is called "Forensics-First Termination."

While traditional forensics relies on post-incident investigation and reconstruction, VCD has the system autonomously preserve evidence before self-destruction. Before attackers have an opportunity to destroy evidence, the following information is recorded in persistent storage outside the component:

- Source IP address and session information
- Type and content of detected anomaly (request/query immediately before detection)
- High-precision timestamp
- Component memory state, stack trace, and message queue state

In runtimes such as Elixir/OTP, VM internal state including process memory, reduction counts, and scheduler information can also be captured as forensic information. This enhances the reproducibility of the attack's intrusion path and scope of impact.

Critical to Forensics-First Termination is guaranteeing the order of evidence transmission and component termination. The risk of a component dying before transmission completes is addressed by the critical path minimization described in 3.2: only local file writes are performed before self-destruction, with data transfer delegated to a separate component.

3.4 Separation of Concerns

VCD clearly separates the "responsibility to self-destruct" from the "responsibility to transmit evidence." This separation ensures that evidence transmission can complete even after the self-destructing component has terminated.

Component	Responsibility
Self-destructing component	Detection, file writing, and self-destruction only
Subsequent component	File monitoring, data transfer, and external coordination only

An important principle accompanies this separation: reverse communication from the subsequent component to the self-destructing component is prohibited. By not accepting external instructions or feedback, the self-destructing component's behavior cannot be affected even if the subsequent component or external storage is compromised.

Separation of concerns is closely related to critical path minimization. The self-destructing component can focus solely on "file writing and self-destruction" precisely because heavy subsequent processing is delegated to a separate component.

3.5 Parallel Validation

When multiple input fields exist, sequential inspection causes inspection time to increase linearly with the number of inputs. VCD adopts a design that inspects all input fields in parallel and immediately transitions to the self-destruction flow the moment any one of them detects an attack.

```
Input A ─┐
Input B ─┼─ Parallel inspection — Any detection — Self-destruction flow
Input C ─┘
```

This design has two effects. First, it contributes to critical path minimization by keeping inspection time close to that of a single field regardless of the number of input fields. Second,

by having the first detecting process immediately initiate the self-destruction flow, response begins without waiting for other inspection results.

Critical to parallel inspection is timeout design. Upper time limits are set for each inspection to prevent the entire system from stopping if any inspection is unresponsive. However, the handling of timeouts requires careful design — consideration must also be given to countermeasures against evasion techniques that embed attacks in large data to cause inspection timeouts. The timeout handling policy must be designed according to system requirements.

3.6 Multi-Layer Volatile Architecture

VCD is achievable with a single process layer alone — a process that detects an attack, self-destructs, and recovers functions as volatile defense. Additionally, combining multiple layers of different granularity can realize multi-layer volatile defense. Each layer has a hierarchical structure where the layer above handles failures that the layer below cannot recover from.

Layer	Technology	Recovery Unit	Response Time
L1: Process	OTP Supervisor	Individual process	Milliseconds to seconds
L2: Container	Kubernetes Pod	Container unit	Seconds to tens of seconds
L3: Node	Kubernetes Cluster	Node unit	Minutes

At L1, the process detecting an attack self-destructs and the Supervisor immediately regenerates a clean process. Minor attacks are resolved at L1 without affecting the entire container.

At L2, the entire container is destroyed and regenerated for high-severity attacks that L1 cannot recover from. Whether L1 self-destruction triggers escalation to L2 is determined by attack severity (see Section 3.8).

L3 is processed autonomously by Kubernetes when it detects node failures and is not subject to explicit control by VCD. The design goal of VCD is to prevent reaching L3 through the operation of L1 and L2 multi-layer volatility.

This hierarchical design creates a natural division of roles: "L1 absorbs minor attacks, L2 handles severe attacks." Note that Sections 3.7 onward assume a multi-layer L1/L2 configuration.

3.7 Graceful Volatile Shutdown

When configuring a multi-layer volatile architecture, the impact on requests being processed by lower layers must be considered when an upper layer self-destructs. While an L1 self-destruction affects only the relevant process, an L2 self-destruction affects all processes running beneath it.

VCD defines a four-phase shutdown sequence called Graceful Volatile Shutdown to address this:

```
① The attacked L1 process immediately self-destructs (evidence preservation → exit)
② The L2 container stops accepting new requests
③ Other processes complete existing in-flight requests
④ After all processes complete, the L2 container terminates and is regenerated
```

Phase ② — stopping new request acceptance — is realized by leveraging infrastructure-side mechanisms. The orchestration platform autonomously removes traffic by detecting changes in the application's health check endpoint responses. A design where the application directly manipulates the infrastructure is not adopted.

In Phase ③, requests being processed by other processes in the same L2 container cannot be ruled out as potentially related to the attack. This risk tolerance is configured through the following modes according to system requirements:

Mode	Behavior	Target Use Case
strict	Immediately destroys all processes upon attack detection	High security requirements
graceful	Waits for in-flight requests to complete before terminating	Balance between availability and security
timeout	Waits up to N seconds then forces termination	Adjust risk tolerance via timeout value

The timeout upper limit must be set shorter than the infrastructure-side timeout setting. This ensures that even if the application cannot autonomously complete shutdown, the infrastructure-side safety net forcibly terminates the L2 container.

3.8 Severity-Based Branching

When an attack is detected, it is not always necessary to proceed to L2 self-destruction. By branching based on attack severity — completing at L1 only or escalating to L2 — unnecessary L2 restarts are avoided while achieving appropriate response.

Severity	Self-destruction level	Example criteria
Low	L1 only	Single XSS/SQLi detection
High	L1 → L2	Multiple attempts from same IP in short time; destructive SQL detection

Severity criteria are designed according to system requirements. The implementation in this paper adopts a method of counting attack attempts from the same IP within a time window and judging as high when the threshold is exceeded (see Section 4 for details).

Critical to severity assessment is that the assessment logic itself must not affect the critical path. VCD recommends keeping the assessment logic lightweight and delegating detailed analysis to a separate component.

3.9 Data Flow Design

Data flow in VCD follows a unidirectional principle. Only the direction from L1 components to external evidence transmission is permitted; reverse communication from external sources to L1 components is prohibited. This principle ensures that L1 component behavior cannot be affected even if external components are compromised.

```
L1 component → File write (before self-destruction)
                ↓
Subsequent component → Data transfer (after detection, asynchronous)
                ↓
External storage → Persistence, blocklist coordination, network layer enforcement
```

Component	Role	Communication direction
L1 component	Detection, evidence writing, self-destruction	→ File only
Subsequent component	File monitoring, data transfer	File → External storage
External storage	Evidence persistence, blocklist management	→ Network layer
Network layer	Source IP blocking	None (receive only)

Network layer blocking prevents attacker connections from reaching the L1 component. Combined with L1-layer blocking, this realizes a multi-layer blocking mechanism.

Since data transfer to external storage is performed asynchronously, evidence is not lost even if the L1 component restarts before transfer completes. The file write completes synchronously before the L1 component self-destructs, and the subsequent component can read it afterward.

4. Implementation

4.1 Implementation Approach

This section demonstrates that the VCD paradigm and architecture defined in Section 3 are technically feasible. Implementation proceeded in three phases, with each phase inheriting the proof of concept from the previous phase toward a more complete realization of VCD.

Phase	Technology	Primary demonstration
Phase 1	Python / Flask	Basic VCD self-destruction, recovery, and blocking behavior
Phase 2	Elixir / OTP	Process-level volatility via OTP
Phase 3	Elixir + Kubernetes	Multi-layer volatility and Graceful Volatile Shutdown

All phase implementations are available in a public repository. This implementation is a proof of concept demonstrating that the VCD concept is technically viable, not production-quality software. Detection accuracy, performance optimization, and comprehensive error handling are left as future work.

4.2 Phase 1: Proof of Concept (Python / Flask)

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase1/python

Phase 1 implemented VCD's basic behavior in a minimal configuration using Python and Flask. The goal was to verify that the five steps — detection, evidence preservation, self-destruction, recovery, and blocking — operate as a continuous flow.

Implemented features include:

- Regex-based XSS and SQLi pattern detection
- Appending detection information to a local file (append-only)
- Immediate self-destruction via os._exit(1)
- Automatic restart via supervisord
- Blocklist loading on startup and 403 responses to blocked IPs

Python was chosen because simplicity of implementation was prioritized at the proof-of-concept stage. The fact that VCD's core can be realized in Python, which lacks OTP's process management mechanisms, demonstrates that VCD is a universal paradigm not dependent on specific languages or runtime environments.

4.3 Phase 2: Elixir / OTP

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase2/elixir

Phase 2 implemented VCD using Elixir and OTP, achieving the implementation most aligned with VCD's design philosophy. The key difference from Phase 1 is that process management, state management, and parallel processing are supported at the runtime level.

Implemented features include:

- HTTP request handling via Plug + Cowboy
- Parallel inspection of all input fields via Task.async_stream
- Blocklist management via GenServer + ETS (state persists across process restarts)
- Immediate self-destruction via exit(:attack_detected)
- Automatic restart via OTP Supervisor
- Forensic information writing including VM internal state (memory, reduction count, stack trace) before self-destruction

Phase 2 was implemented with Plug alone without using Phoenix. VCD's PoC requires only HTTP request handling and detection logic, and a full-stack framework was judged excessive. This choice was also appropriate from the perspective of simply demonstrating VCD's core.

4.4 Phase 3: Elixir + Kubernetes

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase3/k8s

Phase 3 integrated Kubernetes into Phase 2's implementation, realizing L1/L2 multi-layer volatility and Graceful Volatile Shutdown.

Features added or extended in Phase 3 include:

- L1/L2 branching management via ShutdownState GenServer
- Kubernetes self-destruction notification via Readiness Probe endpoint
- Severity assessment (escalate to L2 when attack count from same IP exceeds threshold)
- Three Graceful Volatile Shutdown modes (strict, graceful, timeout)
- Forensic file sharing with sidecar container via emptyDir volume
- File monitoring via inotify and data transfer by sidecar container
- Periodic NetworkPolicy updates by sidecar (IP blocking at network layer)

L2 self-destruction notification was realized using the Readiness Probe. When ShutdownState determines L2 self-destruction, the Readiness Probe endpoint switches to returning 503, and Kubernetes automatically removes the Pod from Service endpoints. A design where the application directly manipulates the Kubernetes API was not adopted.

Forensic information transfer and NetworkPolicy updates are handled by the sidecar container. The application container only writes to files, and the sidecar detects file changes via inotify and takes over subsequent processing. There is no feedback from the sidecar to the application container, with unidirectionality guaranteed by Kubernetes NetworkPolicy.

5. Discussion

5.1 False Positive Problem

VCD's self-destruction immediately interrupts request processing, so false positives have a significant impact on legitimate users. In particular, if L2 self-destruction is triggered by a false positive, the entire container restarts causing a service outage.

This implementation adopts regex-based detection of XSS and SQLi, but cyberattacks are not limited to these types. Attacks that do not fit known patterns — such as command injection, path traversal, buffer overflow, and zero-day attacks — cannot be detected. Furthermore, the question of "what constitutes an attack" is itself an unresolved problem, and VCD's effectiveness is strongly dependent on detection logic accuracy. This paper does not delve into detection logic details and positions this as a future research topic.

Designs to minimize the impact of false positives include:

- Refined severity assessment (do not escalate to L2 for single detection events)
- Migration to semantic detection using machine learning
- Introducing manual confirmation flows for suspected false positives

5.2 Limitations of IP-Based Blocking

This implementation's blocking mechanism is based on source IP addresses. This approach has several limitations.

First, when multiple users exist behind NAT or proxies, legitimate users sharing an IP with an attacker may be affected. Second, attackers can easily evade blocking by changing their IP address. Third, for distributed attacks from large numbers of IP addresses (DDoS), the blocklist grows without bound.

Addressing these limitations requires designing a blocking mechanism that combines IP addresses with other identifying information such as session IDs, user agents, and behavioral patterns.

5.3 Atomicity of Self-Destruction

VCD's critical path is the two steps "file write → exit," but if the process terminates abnormally mid-write, there is a risk of forensic information remaining in an incomplete state in the file.

An atomic file write design is effective against this risk. Specifically, a method of renaming after completing a write to a temporary file is viable. Since rename operations in file systems are atomic, only complete entries are guaranteed to exist when the subsequent component reads the file.

5.4 Evidence Preservation When Sidecar Fails

If the sidecar container stops due to failure, forensic information data transfer may not be performed. Files written by the application container are stored in an emptyDir volume, but emptyDir contents are lost if the Pod restarts.

Countermeasures for this problem include:

- Storing forensic files in persistent volumes rather than emptyDir
- Deploying the sidecar as a Kubernetes DaemonSet, independent of the application Pod lifecycle
- Strengthening sidecar health checks for immediate detection and restart on failure

5.5 DoS Attacks Exploiting the Self-Destruction Mechanism

VCD's self-destruction mechanism may be exploited by attackers. DoS attacks that intentionally send attack patterns to repeatedly self-destruct processes and containers, causing service outages, are conceivable.

This problem is a fundamental design tradeoff in VCD. Increasing self-destruction speed increases DoS vulnerability, and strengthening DoS countermeasures slows self-destruction decisions. Countermeasures include:

- Setting an upper limit on self-destructions within a time period, stopping self-destruction and issuing warnings when exceeded
- Setting a high L2 self-destruction threshold so simple attack patterns do not reach L2
- Combining upper-layer rate limiting with VCD

5.6 Future Work

This implementation is an application-embedded VCD, requiring application modification for deployment to existing systems. To achieve wider adoption, realizing a proxy-type VCD that can be deployed in front of existing systems is effective. Proxy-type VCD can apply volatile defense without modifying applications, enabling deployment regardless of language or framework.

Applying VCD to proxy layers such as nginx and Envoy, and to firewall and network layers beyond the application layer, is also a future research topic. Demonstrating VCD functioning at each layer would show that VCD is a universal defense paradigm not dependent on specific implementations.

Improving detection accuracy is also an important challenge. This implementation adopts regex-based detection, but migrating to semantic detection using machine learning would reduce false positive rates and enable response to more sophisticated attack patterns.

A fundamental design challenge for VCD is handling processes that must not be self-destructed. Processes where mid-stream self-destruction is not business-acceptable — such as payment processing, long-running transactions, and stateful sessions — exist. While mitigation may be possible through separation into proxy-type or functional responsibility division, this aspect is fundamentally incompatible with VCD's premise that "all processes can self-destruct," and future research is needed.

6. Conclusion

This paper proposed Volatile Cyber Defense (VCD) as a new defense paradigm addressing the contaminated state persistence problem after intrusion.

The core of VCD is the design principle of "intentionally dying upon detection and cleanly reviving." A component that detects an attack immediately preserves evidence, self-destructs, and autonomously recovers in a clean state, structurally eliminating the attacker's opportunity to exploit the contaminated state. This philosophy transfers Erlang/OTP's "Let it crash" philosophy to the security context, and rests upon the disposal and regeneration infrastructure provided by cloud-native era infrastructure.

The contributions of this paper are summarized in three points:

- We formalized the contaminated state persistence problem and systematized it as VCD
- We clarified VCD's uniqueness through comparison with MTD, Phoenix Servers, and Intrusion Tolerance
- We demonstrated the technical feasibility of VCD through a multi-layer volatile implementation using Elixir/OTP and Kubernetes

VCD is not in opposition to existing security paradigms; it takes effect by combining with existing layered defense as a layer responsible for "defense after the first line of defense is breached." Within the Defense in Depth framework, VCD is positioned as a new layer.

Future challenges include improving detection logic accuracy, realizing proxy-type VCD, expanding application to proxy and network layers, and handling processes that must not self-destruct. Through demonstrating and resolving these, we will show that VCD is a universal defense paradigm not dependent on specific implementations.

A promising direction for VCD's development is a honeypot transition feature that dynamically diverts attackers to an isolated fake environment instead of self-destruction. Rather than self-destructing and recovering to a clean environment upon attack detection, seamlessly switching to a fake environment where attacker behavior can be observed would further deepen forensic information collection. This feature lies in the extension of VCD's "evidence preservation" philosophy and is one promising direction for development as an open-source software.

References

[1] Jajodia, S., Ghosh, A. K., Swarup, V., Wang, C., & Wang, X. S. (Eds.). (2011). Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats. Springer.

https://link.springer.com/book/10.1007/978-1-4614-0977-9

[2] Fowler, M. (2012). PhoenixServer. martinfowler.com.

https://martinfowler.com/bliki/PhoenixServer.html

[3] Veríssimo, P. E., Neves, N. F., & Correia, M. P. (2003). Intrusion-Tolerant Architectures: Concepts and Design. In Architecting Dependable Systems, LNCS vol. 2677, pp. 3–36. Springer.

https://link.springer.com/chapter/10.1007/3-540-45177-3_1

[4] Kent, K., Chevalier, S., Grance, T., & Dang, H. (2006). Guide to Integrating Forensic Techniques into Incident Response. NIST Special Publication 800-86.

https://csrc.nist.gov/pubs/sp/800/86/final

[5] Armstrong, J. (2003). Making Reliable Distributed Systems in the Presence of Software Errors (Doctoral dissertation). KTH Royal Institute of Technology.

https://erlang.org/download/armstrong_thesis_2003.pdf

[6] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 50–57.

https://dl.acm.org/doi/10.1145/2890784